
Countersign: Evidence-Grounded Supervision of Coding-Agent Completion Claims

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents terminate when they believe a task is done, but that belief is not
2 always supported by the evidence in their own execution trace. We present *Countersign*,
3 a deterministic supervisory meta-agent that audits a worker’s completion
4 claim against the trace—checking that cited tests exist, remain fresh with respect
5 to the files they cover, postdate the latest requirement update, and do not rest on
6 superseded sources—and can block termination and demand fresh evidence. Countersign
7 is deliberately *trace-only*: it judges whether a claim is *justified* at claim time,
8 a different question from whether the implementation is *correct*, which a hidden
9 evaluator answers once after termination. In a preregistered, frozen campaign
10 of 490 real-model runs over ten held-out fixtures, Countersign issued 17 blocks;
11 all 17 landed on proposals an independent oracle also labelled unsupported, and
12 none on any of the 314 supported proposals. A counterfactual replay shows a
13 conventional continuous-integration gate catches 2 of 15 unsupported proposals
14 where Countersign catches 11 of 11 it judged. Our negative results are equally
15 informative: truncating the worker’s visible history did not increase unsupported
16 completion, the effect appearing only when history is *replaced by a narrative*
17 *summary*, and we decompose that effect into prompt and gate contributions. We
18 also quantify the construct honestly—25 of 27 unsupported proposals were backed
19 by work that passed hidden evaluation—so Countersign is an evidence-hygiene
20 instrument whose safety value remains a hypothesis under test.

21 1 Introduction

22 An autonomous coding agent ends its episode by asserting completion. That assertion is a claim about
23 evidence: the tests it ran, the edits it made, the requirements it was given. When the claim outruns
24 the evidence—citing a test that predates the final edit, or resting on a document the project has since
25 superseded—the agent has not necessarily produced wrong code, but it has produced an *unwarranted*
26 report, and any human or downstream system that trusts the report inherits a risk it cannot see.

27 We study whether a second agent, given authority over the first agent’s termination, can reliably tell
28 warranted completion claims from unwarranted ones, and what that authority costs. Our supervisor,
29 Countersign, reads only the worker’s execution trace: it never consults ground truth, and the hidden
30 evaluator that determines correctness runs exactly once, after termination, identically in every
31 condition. This separation is the paper’s organizing principle. A claim can be justified and wrong; it
32 can also be correct and unjustified. Conflating the two is what makes “the verifier improved outcomes”
33 an unfalsifiable statement.

34 **Contributions.** We contribute (i) a supervision loop with halting authority over a worker’s termination,
35 scored against an independent post-hoc oracle it does not compute (§2); (ii) a preregistered evaluation
36 on matched fixture pairs and negative controls that price false blocks, under a frozen protocol with

37 a public deviation ledger (§3); (iii) empirical results across 490 runs—perfect block precision, a
38 counterfactual ablation against a conventional CI gate, a null dose–response under truncation, and a
39 regime-specific effect under summarization decomposed into prompt and gate contributions (§4); and
40 (iv) a calibrated account of the gate as an evidence-hygiene instrument rather than a demonstrated
41 safety mechanism (§5).

42 2 The Countersign Supervisor

43 **Worker and ledger.** The worker is a tool-loop coding agent (self-hosted open-weight models) in a
44 git-backed workspace. Every action appends to an *evidence ledger*: tool calls with outcomes, file
45 mutations, test runs with inferred coverage, and requirement updates. To finish, it emits a *finish*
46 action carrying a claim and a list of cited evidence identifiers.

47 **The audit.** Countersign checks four properties of the cited evidence: (i) at least one cited event is a
48 successful test run; (ii) no cited test predates a later mutation of a file it covers (coverage is inferred
49 statically, so documentation edits do not invalidate code evidence, and a failure superseded by a
50 later passing run does not permanently contradict the claim); (iii) cited evidence postdates the latest
51 relevant requirement update; and (iv) the claim does not rest on sources the task marks superseded.
52 Freshness is tri-state—the supervisor abstains rather than guessing when coverage is unknown.

53 **Authority.** On failure the supervisor blocks termination and returns the specific deficiency (“your
54 cited test predates the edit to X”). A repair variant additionally issues the smallest evidence-refreshing
55 action. Every arm logs the *raw* verifier decision separately from the *enforced* one, so non-blocking
56 arms still measure what the supervisor would have done.

57 **Independent oracle.** A post-hoc oracle labels every proposal *supported/unsupported/uncertain*
58 from fixture-authored completion policies and canonical trace chronology. It shares no code with the
59 online verifier; the two may disagree, and §4 reports a regime in which they do.

60 3 Evaluation Design

61 **Fixtures.** Ten held-out fixture repositories, authored to a preregistered design and frozen before any
62 real-model run was inspected. Three *matched pairs* isolate one failure mode each under context
63 parity (identical planned actions; workspaces byte-identical but for one designed file): *temporal*
64 (cited tests predate the final edit), *provenance* (fresh evidence grounded in a superseded document),
65 and *requirement* (a clarification arrives after the last passing run). Four *negative controls* contain no
66 trap—documentation edits after green tests, an unrelated-module edit with a targeted-test citation,
67 a legitimate zero-change audit, and an irrelevant late clarification—so any block on them is a false
68 positive by construction. A separate development suite, on which the rules were iterated, is never
69 used for evaluation.

70 **Conditions.** Arms cross a worker prompt variant with verifier authority: *memory_baseline*
71 (naive prompt, no gate), *observe_only* (evidence-citation prompt; verifier computes and logs a
72 decision but never blocks and never returns feedback to the worker), *verification_only* (same
73 prompt, blocking), *verification_and_repair*, and an evaluation-only *oracle_supervisor* upper
74 bound. Because *memory_baseline* and *verification_only* differ in *both* prompt and gate, we never
75 report that contrast as a gate effect; *observe_only* is the prompt-matched control, and we verified
76 in-trace that it emits zero worker-visible feedback events.

77 **Memory degradation.** We additionally run the fixtures under a frozen registry of degradation profiles
78 applied only to *model-visible* state: graded *lossy truncation* of visible history (low/medium/high) and
79 a *resume-summary* regime in which the visible ledger is replaced by a single truthful summary—the
80 pattern deployed scaffolds use on session resumption. The canonical ledger, verifier, oracle, and
81 evaluator always see undegraded state.

82 **Protocol.** Temperature 0.7, sampler seeds 0–4 (greedy decoding would make multi-seed replication
83 pseudoreplication), action budget 24. Endpoints, comparisons, and interpretation branches for both
84 positive and null outcomes were committed before the runs; the frozen protocol hashes code, fixtures,
85 and model weights and refuses to start on a dirty tree.

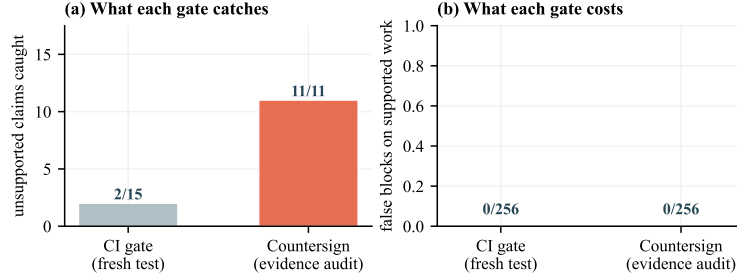


Figure 1: Countersign versus the conventional CI rule “require a successful test run after the last edit,” replayed counterfactually over every stored proposal. (a) The CI rule catches 2 of 15 oracle-unsupported claims; Countersign catches all 11 it judged. The 10 it catches alone are cases where the tests *were* fresh and the deficiency was citation or provenance—which test-freshness gating cannot observe. (b) Neither gate blocked any of the 256 supported proposals: the difference is blindness, not conservatism.

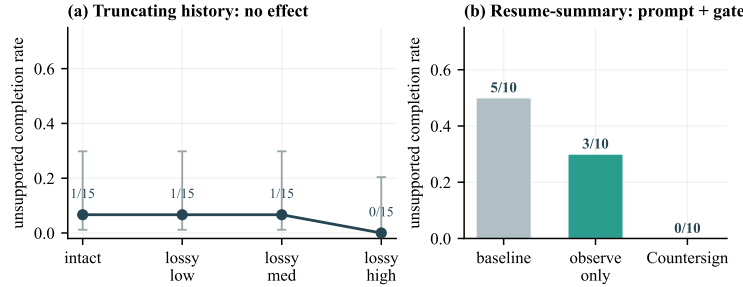


Figure 2: Memory degradation does not act uniformly. (a) Graded truncation of the worker’s visible history leaves unsupported completion flat (Wilson 95% intervals; counts annotated). (b) Replacing history with a resume summary does produce the effect, and it decomposes: the evidence-citation prompt accounts for 2 of the 5 cases and the gate for the remaining 3. Small n ; we report the decomposition, not a significance claim.

86 4 Results

87 All numbers are computed from audited run manifests; the worker is `qwen2.5-coder:14b` unless
 88 noted. The campaign comprises 490 real-model runs yielding 341 oracle-judged finish proposals, of
 89 which 27 are labelled unsupported.

90 **Blocks are precise.** Across every arm in which the verifier ran, Countersign issued 17 blocks. All
 91 17 landed on proposals the independent oracle also labelled unsupported; **none** landed on any of
 92 the 314 supported proposals (block precision 1.00 against the oracle). On the four negative controls
 93 specifically—where every finish is legitimate by construction—18 proposals produced 2 raw would-
 94 blocks, and the oracle independently labelled both of those proposals unsupported, so the measured
 95 false-block count on oracle-supported work is zero.

96 **A conventional CI gate is blind to what actually occurs.** We replay every stored proposal against the
 97 standard-practice rule “require a successful test run after the last edit,” a strict subset of Countersign’s
 98 temporal check (Figure 1). **Ten of the fifteen unsupported proposals are caught only by the**
 99 **evidence rules**, because in those cases the tests were fresh and the deficiency lay in citation or
 100 provenance.

101 **Truncating memory does not induce false claims.** Under the graded lossy regime, baseline
 102 unsupported completion on trap fixtures was flat across severity (Figure 2a). The preregistered null
 103 branch applies: this worker’s re-verification habits survive history truncation, and we report the
 104 finding as robustness rather than reframing it.

105 **The effect is regime-specific, and decomposes.** Replacing history with a resume summary—rather
 106 than truncating it—does produce unsupported completion (Figure 2b). Read as a 2×2 over ten
 107 matched cells, the prompt accounts for 2 of the 5 cases and the gate for 3 (exact McNemar $p = 0.50$
 108 and $p = 0.25$; combined $5 \rightarrow 0$, $p = 0.0625$). We therefore report a decomposition and claim no
 109 significant effect. An a-priori power analysis explains why: with all discordant pairs in one direction
 110 the exact test requires $b \geq 6$ for $p < .05$, so the campaign is one discordant pair short of significance
 111 by construction, and 16 matched cells would be needed for 80% power at the observed effect.

112 **Measurement without power is the cleanest evidence.** In `observe_only` the verifier judges but
 113 cannot act, so its decisions cannot have influenced the trajectories it scores. Its would-block fired in
 114 exactly the 3 unsupported cells and none of the other 7 (TP= 3, FP= 0, FN= 0)—evidence immune
 115 to the circularity objection a blocking arm invites.

116 **What “unsupported” means here.** 25 of the 27 unsupported proposals had a fresh passing test in
 117 the canonical trace and passed hidden evaluation; 18 of 27 are unsupported because the worker cited
 118 *no* evidence at all. The measured phenomenon is therefore predominantly *unsubstantiated* rather than
 119 *incorrect* completion—verified work the worker failed to point at, most often when summarization
 120 removed the identifiers it would have cited.

121 5 Supervisor-Failure Analysis and Limitations

122 **The structural blind spot.** The provenance pair is built so that evidence is fresh but grounded in a
 123 superseded document. A trace-only supervisor allows those finishes, and the hidden evaluator fails
 124 them—a supported-but-incorrect outcome that no amount of trace auditing can reach. This bounds
 125 what runtime evidence supervision can promise.

126 **Two further failure modes** appear in Appendix A: the online classifier and the oracle diverge under
 127 heavy degradation (the classifier reads the worker’s degraded view, the oracle the canonical trace),
 128 and one fixture family is censored by task weight rather than action budget.

129 **Limitations.** First, and most important, the held-out traps and the supervisor’s rules were designed by
 130 the same authors; the development/held-out split prevents rule tuning on evaluation fixtures but cannot
 131 remove the conceptual correlation between what we chose to trap and what we chose to detect. Two
 132 design features partially offset this—the provenance family is built to *defeat* the supervisor, and the
 133 negative controls price the direction a self-built evaluation is least motivated to measure—but partial
 134 offset is not elimination. Second, one worker model on ten small fixture repositories establishes
 135 nothing about prevalence in production software, and our own null makes that gap sharper. Third,
 136 no result reaches $p < .05$: we report decompositions and intervals, not significance. Fourth, the
 137 degradation profiles are our own transforms, not a deployed scaffold’s condenser. Fifth, oracle-label
 138 validation covers a single-rater sample.

139 6 Related Work

140 Prior work measures workers gaming outcome checks METR [2025] and proposes runtime guardrails
 141 that gate *actions* for safety Chennabasappa et al. [2025]; Countersign instead gates *termination* for
 142 evidential support and prices the resulting false blocks. Self-verification asks the worker to check
 143 itself Shinn et al. [2023]—external supervision is the complement when self-checking is what is
 144 in doubt. Outcome and process verifiers Cobbe et al. [2021], Lightman et al. [2024] inform our
 145 split between an online process check at termination and a single post-termination outcome check;
 146 long-context degradation Liu et al. [2024], Packer et al. [2023] motivates the memory regimes, and
 147 hidden-evaluation benchmarks Jimenez et al. [2024] the correctness oracle.

148 Responsible-Use Statement

149 Countersign controls agent overclaiming, and we expect its primary societal effect to be positive—
 150 fewer unverified “done” states in automated software work—but agents supervising agents carries
 151 risks this paper keeps visible.

152 **An allow is not a guarantee.** The supervisor audits justification, not correctness: our provenance
 153 fixtures construct cases where fresh, well-cited evidence grounded in the wrong source passes the

154 audit while the hidden evaluator fails the work, and 25 of 27 unsupported proposals concerned work
155 that was in fact verified. Countersign is an evidence-hygiene instrument and must never be marketed
156 as verification of correctness.

157 **Oversight displacement.** A visible gate invites humans to stop checking. Because the supervisor is
158 deterministic its failure modes are enumerable and we report them alongside its precision; verdicts
159 should reach operators as evidence audits with cited events, not pass/fail badges.

160 **Who supervises the supervisor.** Halting authority creates a new failure surface—over-blocking
161 denies service, repair guidance can loop a worker into budget exhaustion—which we price directly
162 and recommend as standard counter-metrics for any deployed gate.

163 **Scope.** Experiments use self-hosted open-weight models on fixture repositories: no human subjects,
164 personal data, or production systems. The released artifact contains fixtures, protocols, and run
165 records only.

166 References

167 Sahana Chennabasappa et al. LlamaFirewall: An open source guardrail system for building secure
168 AI agents. *arXiv preprint arXiv:2505.03574*, 2025. Author list to be verified against the arXiv
169 page before submission.

170 Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser,
171 Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John
172 Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*,
173 2021.

174 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik
175 Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International
176 Conference on Learning Representations (ICLR)*, 2024.

177 Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan
178 Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International
179 Conference on Learning Representations (ICLR)*, 2024.

180 Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and
181 Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the
182 Association for Computational Linguistics*, 12, 2024.

183 METR. Recent frontier models are reward hacking. [https://metr.org/blog/
184 2025-06-05-recent-reward-hacking/](https://metr.org/blog/2025-06-05-recent-reward-hacking/), 2025. PLACEHOLDER: verify before submission.

185 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E.
186 Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*,
187 2023.

188 Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion:
189 Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing
190 Systems (NeurIPS)*, 2023.

191 A Additional Failure Modes

192 **Classifier/oracle divergence.** Under high degradation the online claim classifier disagreed with the
193 independent oracle, because the classifier reads the worker’s degraded view while the oracle reads the
194 canonical trace. All endpoints in the main text are oracle-anchored; the divergence is evidence that an
195 independent oracle is necessary rather than redundant.

196 **Task weight censors a family.** Requirement-family trap runs exhaust the action budget in 3/5 runs
197 in every arm, and raising the budget from 24 to 40 actions left that rate unchanged. The family is
198 intrinsically too heavy for this worker—a fixture-design finding, not a budget-calibration error.

199 **B Reproducibility**

200 The campaign spans three tags of one freeze: a base freeze plus two infrastructure-only fixes applied
201 mid-campaign (interruption-recovery artifact reuse; a tool-level crash on syntactically invalid worker-
202 written code surfacing as a tool error rather than killing the episode). The hashed verifier-policy files
203 are byte-identical across all three tags: no verifier, oracle, or fixture logic changed after the freeze.
204 Every deviation, interim inspection, and protocol-identifier supersession—including two interim
205 analyses that informed spending decisions, and one corrected over-interpretation—is recorded in the
206 deviation ledger shipped with the artifact. The artifact is a relocatable bundle whose integrity audit
207 passes from the copied location.